

Security Code Review Skills

A human-triggered agent that security-audits **both sides of a software modernization** and reports what the migration did to the security posture.

Run it, answer three questions, and get three self-contained HTML reports in one folder:

Report	Answers
Legacy	What was wrong before the migration? (<i>diagnostic baseline — no remediation</i>)
Modernized	What is wrong now, and how do we fix it without breaking anything?
Comparison	What did the migration resolve , carry across , and introduce ?

```
/security-code-review
```

Before I start, three things:

1. Project name?

(used for the report folder and titles)

2. Legacy source path?

(leave blank or say "none" if there is no legacy codebase)

3. Modernized source path?

(required)

Install

```
.\install.ps1 # Windows -> ~\.claude\skills
```

```
./install.sh # macOS / Linux / Git Bash -> ~/.claude/skills
./install.sh --project /src/acme # or into one project
```

Start a new Claude Code session and the three skills appear in `/help`.

Zero dependencies. No pip, no npm, no Docker, no scanner binaries. The installers only copy files. If anything ever asks you to install something to run an audit, that is a bug.

Prefer a form to a prompt?

```
node ui/server.js --open # http://127.0.0.1:7099
```

An optional dark web UI that collects the three inputs, validates the paths before you spend anything, runs the review and links the reports. Node standard library only — still no npm. It is the one part of this project that needs Node; the skills themselves stay dependency-free. See [ui/README.md](#).

What makes the comparison worth reading

Comparing two audits is not a diff — the codebases are in different languages with different layouts, and **nothing joins on file path**. Findings are paired on *weakness class* × *functional area*, every pairing publishes its confidence, and results land in five buckets:

Introduced	created by the modernization – the legacy system got this right
Inherited	present before and after; the rewrite carried the flaw across
New surface	new architecture, new exposure – not a regression, but real
Resolved	gone in the modernized tree ... but see below
Not comparable	no counterpart existed to compare against

Two distinctions carry most of the value:

introduced vs **new-surface**. A green-screen 4GL app has no CSRF risk and no CORS policy. Finding those gaps in the web rewrite is the cost of the new architecture, not a regression. Filing every modernized-only finding as "introduced by the migration" produces a document that reads as an indictment and gets dismissed for that reason.

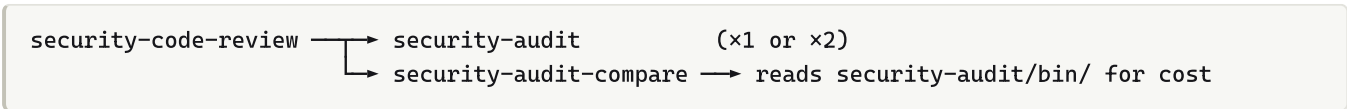
resolved is not automatically good news. A vulnerability that disappeared because the feature carrying it was never migrated has not been fixed — it is an open migration item, and it returns with the feature. Those are pulled into their own **coverage risks** section, because "12 resolved" otherwise reads as twelve fixes.

Documentation

Document	Contents
Agent Design	Architecture, flow diagrams, why a skill rather than a subagent, the comparison model, design constraints
Skills Reference	All three skills: options, procedures, rule packs, matching rules, file layouts
Report Structures	Section-by-section for the Legacy, Modernized and Comparison reports
Install and Usage	Installing, running, options, cost, troubleshooting, embedding
Web UI	The optional local front end — requirements, endpoints, security model

Original design notes for the audit engine are in [docs/Skill docs/](#).

The three skills



Skill	Role
<code>security-code-review</code>	Asks, validates, confirms, sequences the phases, announces

Skill	Role
<code>security-audit</code>	Audits one source tree → one HTML report + <code>findings.json</code> . Works standalone on any codebase
<code>security-audit-compare</code>	Joins two audits → the comparison report

Keep them as siblings — `security-audit-compare` reads `security-audit`'s cost collectors and pricing table by relative path, so there is one pricing table rather than two that drift apart.

Design commitments

These are properties to preserve, not incidental behaviour:

- **Human-triggered only.** No scheduling, no watch mode. Each run costs real money.
 - **Validate, confirm, then spend.** A mistyped path is caught while it is still free.
 - **Read-only.** The system audits and reports; it never edits either codebase.
 - **Evidence or it does not ship.** Every finding cites `file:line` with a real excerpt.
 - **No invented identifiers.** CWE/ATT&CK/CVE numbers are omitted rather than guessed.
 - **Never re-score during comparison.** Each audit owns its numbers; the comparison joins them.
 - ****Report what was *not* done.**** Unassessed categories, skipped directories, unmapped areas, tentative pairings, uncollected costs — all stated in Limitations.
 - **Cost in every report,** in tokens and dollars, and never an estimate presented as a measurement.
-

Standards

Findings map to OWASP Top 10 (2021, noting 2025 where it differs), CWE, MITRE ATT&CK, and NIST SP 800-218 (SSDF) / SP 800-53 Rev. 5. Scoring weights are fixed and published in `security-audit/references/scoring.md` — the report shows the arithmetic so a reader can recompute it by hand.